

Technical Report: File Explorer

Project: File Manager in C++

Overview

During the development of my Simple File Explorer, I encountered several critical configuration, linking, and compatibility issues. Because my project relies on modern C++ system-level libraries to communicate with the Operating System kernel, ensuring my development environment and code architecture were perfectly aligned was a major technical hurdle. Below is a detailed breakdown of the primary challenges I faced during development and the exact steps I took to resolve them.

Challenge 1: Outdated Compiler and C++17 Standard Compliance

- **Situation (What were the challenges):** The core of my application relies heavily on the `<filesystem>` library to interact with the OS (e.g., executing system calls like `fs::directory_iterator` and querying metadata). However, upon my first build attempt, the IDE threw fatal errors stating that the `std::filesystem` namespace did not exist. I discovered the root cause was that the previous, default version of MinGW bundled with my Code::Blocks was outdated and did not fully support modern C++ standards.
 - **Task (What I was doing previously):** I was attempting to compile my initial codebase—which heavily utilized modern file I/O operations—using the default Code::Blocks environment and an older, incompatible MinGW version.
 - **Action (What I did to address the issue):** I abandoned the outdated default MinGW setup and installed a completely modern MSYS2 64-bit compiler environment (`mingw-w64-x86_64-gcc`). After installing the new compiler, I opened my Code::Blocks `Settings -> Compiler` menu. Under the "Compiler Flags" tab, I explicitly checked the box to force the compiler to follow the modern standard: *"Have g++ follow the C++17 ISO C++ language standard [-std=c++17]"*.
 - **Result (What was the outcome of my action):** By upgrading my compiler and explicitly enforcing the C++17 standard, my MSYS2 environment successfully located and linked the `<filesystem>` library. My application could now successfully interface with the Windows kernel to perform directory iterations and metadata retrieval.
-

Challenge 2: IDE Toolchain Path Corruption

- **Situation (What were the challenges):** After I successfully installed the new MSYS2 MinGW compiler, my Code::Blocks IDE completely failed to build the project. It threw a severe path execution error: `Tried to run compiler executable 'C:\Program Files\CodeBlocks\MinGW\bin\C:\msys64\mingw64\bin\gcc.exe', but failed!` The IDE had corrupted the toolchain path by illegally concatenating the old, default MinGW directory with my newly installed MSYS2 directory.
 - **Task (What I was doing previously):** I was attempting to connect my newly downloaded MSYS2 compiler to the Code::Blocks IDE by clicking the "Reset Defaults" button in the compiler settings, which inadvertently caused a massive path collision.
 - **Action (What I did to address the issue):** I realized I had to manually override the IDE's internal toolchain mapping. I navigated to `Settings -> Compiler -> Toolchain executables`. I completely erased the corrupted string in the *"Compiler's installation directory"* field and replaced it strictly with my correct root directory: `C:\msys64\mingw64`. Since the master directory was now correct, I removed the absolute paths from the sub-program fields (C compiler, C++ compiler, Linker, Debugger), changing them to just their base executable names (`gcc.exe`, `g++.exe`, `gdb.exe`). Finally, I cleared the corrupted build files by running a "Rebuild Workspace" command.
 - **Result (What was the outcome of my action):** Code::Blocks correctly mapped to my new toolchain binaries. I entirely resolved the path collision, and the IDE was able to locate the compiler executables and successfully build my application without any "executable not found" errors.
-

Challenge 3: Monolithic Code and Linking Errors

- **Situation (What were the challenges):** When I first started writing the code, I kept the source code structured as a single monolithic block. My class definitions, standard library inclusions, and main execution loop were all located in one file. When compiled, this caused severe structural confusion. The compiler struggled to resolve my header guards (`#ifndef FILEMANAGER_H`), resulting in undefined reference errors.
- **Task (What I was doing previously):** I was trying to compile my entire project from a single `main.cpp` file, which violated object-oriented programming standards and confused the linker during my build process.
- **Action (What I did to address the issue):** I decoupled my code into a standard, modular C++ architecture across three distinct files.
 1. I created `FileManager.h` exclusively to hold my class declarations, variables, and function prototypes.

2. I created `FileManager.cpp` to hold the actual logic and implementation of those functions.
 3. I cleaned up my `main.cpp` so its only responsibility was handling the user interface loop and instantiating my `FileManager` object.
 - **Result (What was the outcome of my action):** My separation of concerns eliminated the linking errors entirely. The compiler was able to process my header guards correctly, resulting in a clean, stable, and highly modular build that adhered to professional software engineering practices.
-

Challenge 4: File Listing Crashes on Restricted Directories

- **Situation (What were the challenges):** While testing the "List Directory" feature, the application would abruptly crash and shut down whenever I navigated into certain higher-level system folders (like the root `C:\` drive or `System32`). The issue was that the Operating System denied read access to specific hidden or protected files. Because my program wasn't expecting a hard rejection from the OS, it threw an unhandled exception.
- **Task (What I was doing previously):** I was using a basic `fs::directory_iterator(currentPath)` inside a `for` loop to list all files and folders. I naively assumed the OS would automatically let my program read every file path in the directory.
- **Action (What I did to address the issue):** I researched the `<filesystem>` library's error handling and realized I needed to manage OS-level permission errors gracefully. I wrapped my iteration loop in a `try-catch` block to handle `fs::filesystem_error` exceptions. Furthermore, I updated the directory iterator by passing an additional flag: `fs::directory_options::skip_permission_denied`. This effectively told the kernel, "If I don't have access to read a specific file, just skip it and keep giving me the rest of the list."
- **Result (What was the outcome of my action):** The file listing feature became highly robust. Instead of crashing the entire process when hitting a kernel-level permission roadblock, the program safely caught the error, skipped the restricted files, and successfully printed out all the accessible files. This drastically improved the application's overall stability during deep file system traversal.